



UNIVERSITAT
POLITÀCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escola Tècnica Superior d'Enginyeria Informàtica
Universitat Politècnica de València

Documentación del Proyecto

Trabajo Fin de Grado

Grado en Informática Industrial y Robótica

Autor: Lourdes Francés Llimerá

Tutor: Juan Francisco Blanes Noguera

2025-2026

Tabla de contenidos

Tabla de contenidos.....	2
Índice de tablas.....	3
1. Introducción.....	4
1.1. Qué es LARIC.....	4
1.2. Por qué existe el proyecto.....	4
1.3. Contexto	4
1.4. Objetivos.....	5
1.5. Alcance y limitaciones	5
2. Visión general de la arquitectura.....	5
3. Pila de software.....	6
3.1. ROS 2 y DDS.....	6
3.2. Gazebo y TurtleBot3 (entorno de simulación).....	7
3.3. Husarion ROSbot (entorno real)	7
3.4. La pila de navegación: Nav2	7
3.5. El modelo de lenguaje: Llama 3.3 70B	8
3.6. Backends de inferencia: Groq y Ollama.....	8
3.7. LangChain, ReAct y tool calling	8
3.8. El puente entre ROS 2 y el LLM: RAI.....	8
3.9. La pila de la HMI: PyQt5, voz y gettext	9
4. Cómo se interconectan los componentes	9
4.1. Tópicos de ROS 2	9
4.2. Flujo de extremo a extremo	9
4.3. Flujo de localización	10
4.4. Flujo de parada de emergencia	10
4.5. Sincronización de idioma.....	10
5. El agente y sus 7 herramientas	10
6. Diseño de seguridad	11
7. Internacionalización	11
8. Simulación frente a robot real.....	12
9. Decisiones de diseño	12
10. Resultados.....	13

Índice de tablas

Tabla 1. Tópicos principales de ROS 2 en LARIC.....	9
Tabla 2. Herramientas del agente.	11
Tabla 3. Comparativa de los dos entornos.	12

1. Introducción

1.1. Qué es LARIC

LARIC (Language-based Agent for Robotic Interaction and Control) es un sistema que permite controlar un robot móvil mediante lenguaje natural, por voz o por texto, en español o en inglés. El usuario dice “ve al baño” o “avanza un metro y luego gira a la derecha” y el robot lo ejecuta, sin que el operador necesite conocimientos de robótica ni recordar comandos exactos. Un modelo de lenguaje grande (LLM) interpreta la intención y la traduce a una acción de navegación concreta, que el robot realiza de forma segura y conociendo en todo momento su posición en el mapa.

El nombre resume las cuatro ideas del proyecto:

- **Language-based:** Interfaz operada íntegramente mediante lenguaje natural, facilitando una comunicación intuitiva.
- **Agent:** Un agente de inteligencia artificial (LLM) que procesa la información, razona y planifica las acciones a tomar.
- **Robotic Interaction:** Interacción humano-robot bidireccional, proporcionando realimentación continua sobre el estado del sistema.
- **Control:** Traducción de las intenciones del usuario en movimientos físicos precisos y navegación autónoma mediante Nav2.

1.2. Por qué existe el proyecto

La forma más habitual de mandar a un robot móvil ir a un sitio exige conocer su API (tópicos, acciones, sistemas de coordenadas, etc.) o programar a mano un repertorio rígido de órdenes con comprobaciones del tipo “si el texto contiene avanza o adelante, entonces...”. Ese enfoque no escala y resulta inaccesible para un usuario no técnico. La aparición de los LLM con capacidad de razonamiento y de invocación de herramientas (tool calling) abre una vía distinta: que sea el modelo quien comprenda la petición, en cualquier formulación, y elija la acción robótica adecuada.

LARIC explora precisamente ese puente entre la interacción en lenguaje y la navegación autónoma. El reto no es solo entender la frase, sino hacerlo de forma segura y determinista. De ahí que el diseño imponga que todo movimiento pase por una pila de navegación probada (Nav2), que el robot esté siempre localizado y que exista una parada de emergencia que no dependa del modelo.

1.3. Contexto

El proyecto es un Trabajo de Fin de Grado del Grado en Informática Industrial y Robótica, desarrollado en el instituto ai2 de la Universidad Politécnica de Valencia. Se construye sobre RAI (Robotec AI Framework), una biblioteca que conecta ROS 2 con modelos de lenguaje. El desarrollo y la validación se hacen primero en simulación (robot TurtleBot3 en el simulador Gazebo) y después se trasladan a una plataforma física (Husarion ROSbot), compartiendo el mismo código del agente para ambos entornos.

1.4. Objetivos

El objetivo general es construir un sistema que traduzca lenguaje natural en acciones de un robot móvil de forma robusta, segura y portable. Se desglosa en los siguientes objetos concretos:

- Interpretar comandos en lenguaje natural (voz o texto, español o inglés) y traducirlos a la acción robótica adecuada mediante un agente LLM.
- Delegar todo el movimiento en Nav2, de modo que el robot conozca siempre su posición en el mapa y nunca se mueva “a ciegas”.
- Soportar tanto movimientos primitivos (girar, avanzar, retroceder y secuencias) como navegación autónoma a ubicaciones con nombre (navegación semántica).
- Garantizar la seguridad mediante localización obligatoria, serialización de los movimientos y una parada de emergencia que evite por completo al LLM.
- Ofrecer una interfaz de usuario bilingüe con entrada por voz (Push-to-Talk) y texto, y realimentación clara de lo que el robot entiende y ejecuta.
- Lograr portabilidad entre el entorno simulado y el robot real sin cambios en el código del agente, solo mediante conmutadores de configuración.

1.5. Alcance y limitaciones

LARIC se centra en la comprensión de órdenes y la ejecución de navegación, no en la percepción avanzada ni en la manipulación. La localización inicial la fija el operador, no hay autolocalización. La creación del mapa (SLAM) es un paso previo de configuración, no una capacidad en línea del agente.

2. Visión general de la arquitectura

El sistema se compone de dos procesos independientes que se comunican exclusivamente por tópicos de ROS 2. Este desacoplamiento permite ejecutarlos por separado, reiniciar uno sin tumbar el otro e incluso, en el robot real, repartir el cómputo entre el portátil y el propio robot:

- **HMI (`laric_interface.py`):** Interfaz gráfica PyQt5 con Push-to-Talk, entrada de texto, selector de idioma, log de estado con código de colores y botón de parada de emergencia. Captura la voz, la transcribe y publica el texto del usuario.
- **Agente (`agent_logic.py`):** Nodo que recibe el texto del usuario, lo razona con el LLM mediante LangChain, ejecuta la herramienta de Nav2 seleccionada y publica la realimentación.

Dos módulos son compartidos por ambos procesos: **`config.py`** (configuración: conmutadores, endpoints del LLM, mapa semántico, parámetros de movimiento y el system prompt) e **`i18n.py`** (traducción). El flujo general es el siguiente:

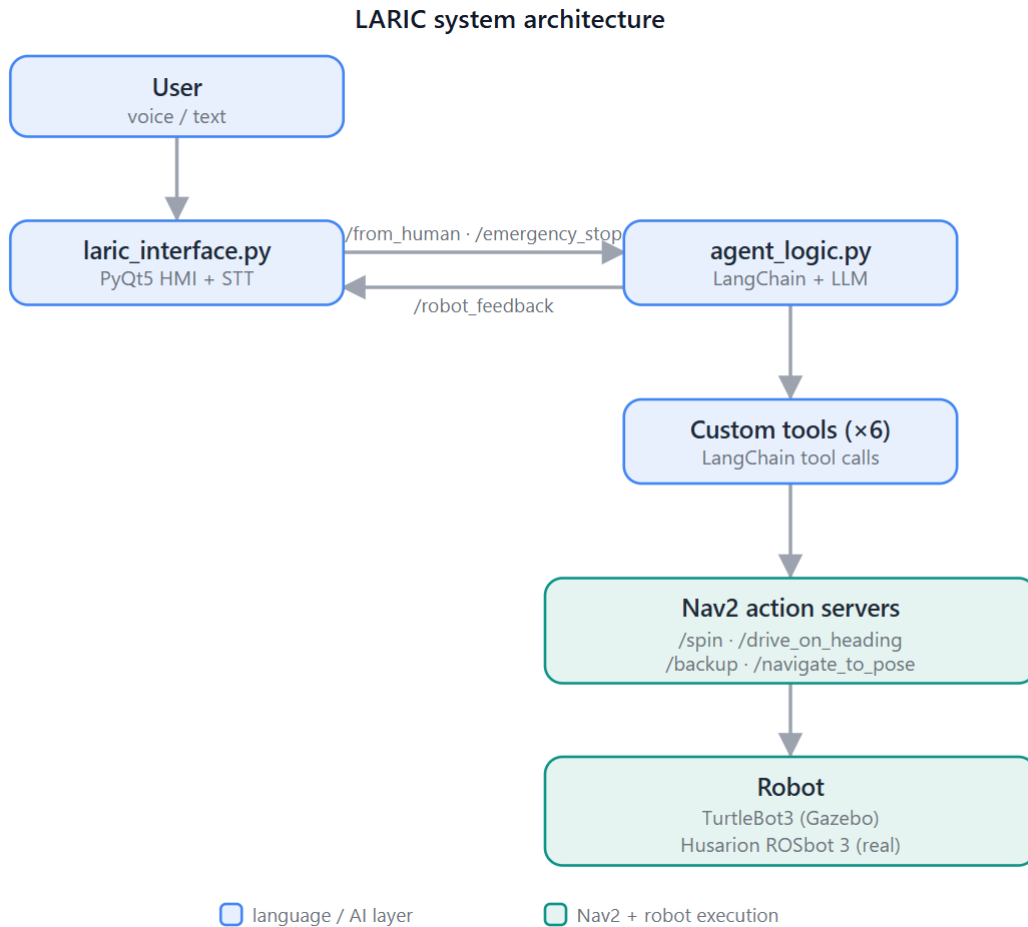


Figura 1. Arquitectura del sistema LARIC.

3. Pila de software

Esta sección describe cada tecnología de la pila y, sobre todo, qué papel cumple dentro de LARIC y cómo se conecta con las demás.

3.1. ROS 2 y DDS

ROS 2 (Robot Operating System 2) es el middleware de robótica sobre el que se apoya todo el sistema. Es un conjunto de bibliotecas y convenciones para que nodos (procesos) heterogéneos se comuniquen. Su transporte es DDS (Data Distribution Service), que aporta un bus publicación/suscripción con descubrimiento automático de nodos y control de calidad de servicio (QoS). ROS 2 ofrece tres patrones de comunicación, y LARIC usa los tres:

- **Tópicos** (publicador/suscriptor, asíncrono): Para el texto del usuario (`/from_human`), la realimentación (`/robot_feedback`), la parada (`/emergency_stop`), el idioma (`/language_changed`), la pose inicial (`/initialpose`) y la velocidad directa (`/cmd_vel`).

- **Acciones** (tareas de larga duración con objetivo, feedback y resultado): Para los movimientos de Nav2 (/spin, /drive_on_heading, /backup, /navigate_to_pose).
- **Servicios** (petición/respuesta): Usados internamente por Nav2.

Otro concepto clave es TF, el árbol de transformadas que relaciona los sistemas de coordenadas del robot (map, odom, base_link, etc.). Su consistencia es lo que permite a Nav2 saber dónde está el robot respecto al mapa. LARIC fija el ROS_DOMAIN_ID a 0 para que el PC y el robot real compartan el mismo dominio DDS.

3.2. Gazebo y TurtleBot3 (entorno de simulación)

Gazebo es un simulador físico 3D. En LARIC carga el mundo turtlebot3_house (una vivienda con habitaciones) y el modelo TurtleBot3 Waffle Pi, un robot diferencial de Robotis con LIDAR 360 y encoders. Gazebo simula la física, los sensores (LIDAR, odometría) y publica el TF, de modo que para el resto de la pila el robot simulado es indistinguible de uno real. El robot aparece (spawn) en una posición alejada de las paredes ($x = -2.0$, $y = -3.0$), un detalle necesario para evitar un falso aviso de colisión de Nav2 en arranque en frío.

3.3. Husarion ROSbot (entorno real)

La plataforma física es un Husarion ROSbot3, un robot móvil diferencial que ejecuta a bordo Ubuntu y ROS 2 Jazzy. A diferencia de la simulación, aquí Nav2 y toda la percepción corren en el propia robot, y el PC actúa solo como cliente ROS 2. El robot distribuye sus componentes como paquetes snap:

- **husarion-rplidar**: Driver del sensor LIDAR.
- **husarion-depthai**: Driver de la cámara de profundidad.
- **husarion-webui**: Interfaz web basada en Foxglove (alojada en http://IP_del_rosbot:8080/ui) para visualizar el mapa y publicar la pose inicial. Sustituye a RViz en el entorno real.
- **Stack de Nav2**: Se arranca con “just start-navigation” y se detiene con “just stop” dentro de ~/rosbot-autonomy.

El mapa del entorno real se crea primero con SLAM (conduciendo el robot) y luego se opera con SLAM desactivado, localizando contra el mapa estático creado.

3.4. La pila de navegación: Nav2

Nav2 es la pila de navegación autónoma de ROS 2 y el corazón de la capa de movimiento de LARIC. Es una decisión de diseño que todo el movimiento pase por Nav2: así el robot opera siempre dentro del marco de navegación, con su posición conocida.

LARIC consume Nav2 de dos maneras: directamente, enviando goals a los behavior plugins para los movimientos primitivos; y a través del action server /navigate_to_pose para la navegación a destinos. Las únicas velocidades crudas que LARIC publica (en /cmd_vel) son las de la parada de emergencia y el gesto de negación.

3.5. El modelo de lenguaje: Llama 3.3 70B

El núcleo de razonamiento es Llama 3.3 70B, un LLM de Meta de 70.000 millones de parámetros. Es el componente que interpreta la intención del usuario y decide qué herramienta invocar y con qué parámetros. Se eligió el modelo de 70B porque modelos más pequeños (por ejemplo, Llama 3.1 8B) fallaban al razonar las reglas de seguridad y llegaban a confundir el signo de ángulos y distancias, algo inaceptable para controlar un robot.

3.6. Backends de inferencia: Groq y Ollama

El mismo modelo de 70B se sirve desde dos backends, seleccionables con el conmutador `is_from_lab` de `config.py`:

- **Ollama del ai2** (`is_from_lab = True`, vía principal): El instituto ai2 mantiene el modelo desplegado en sus servidores, accesible mediante una API de Ollama en su clúster (endpoint definido en la variable URL de `config.py`; modelo `llama3.3:70b`). LARIC no ejecuta el modelo en local, solo envía consultas a esa API. No tiene límite de tokens, pero exige estar en la red de la UPV.
- **Groq** (`is_from_lab = False`, respaldo desde fuera del laboratorio): Inferencia en la nube de baja latencia con el modelo `llama-3.3-70b-versatile`. Su capa gratuita tiene un tope de 100.000 tokens diarios. Requiere la variable de entorno `GROQ_API_KEY`.

3.7. LangChain, ReAct y tool calling

LangChain es la biblioteca que orquesta el ciclo de razonamiento del agente, siguiendo el patrón ReAct (Reasoning + Acting): el modelo alterna entre razonar y actuar (invocar herramientas). En LARIC, `create_tool_calling_agent` y `AgentExecutor` construyen el prompt (el `SYSTEM_PROMPT` más el mensaje del usuario), llaman al LLM y ejecutan la herramienta que el modelo elige mediante tool calling. Cada herramienta declara un esquema de entrada (Pydantic) cuyas descripciones lee el propio LLM para inferir los parámetros. El `AgentExecutor` limita el número de iteraciones para acotar el razonamiento.

El `SYSTEM_PROMPT` (en `config.py`) es la pieza que convierte un comportamiento probabilístico en uno suficiente determinista para controlar un robot. Define las reglas de selección de herramienta, las convenciones de signo de ángulos y distancias, la regla de idioma (responder siempre en el idioma del usuario) y las restricciones de ejecución (una sola herramienta por turno, no auto-cancelarse, cómo evaluar el resultado de una acción).

3.8. El puente entre ROS 2 y el LLM: RAI

RAI (Robotec AI) es la biblioteca que une el mundo del LLM con el de ROS 2. Aporta tres piezas que LARIC usa:

- **ROS2Context:** Decorador que inicializa y limpia `rcipy` (el cliente Python de ROS 2) alrededor del main del agente.

- **ROS2Connector:** Objeto central de comunicación. Métodos principales: `register_callback` (suscribirse a un tópico), `send_message` (publicar), `start_action` / `terminate_action` (gestionar acciones de larga duración).
- **Herramientas de Nav2 preimplementadas:** `NavigateToPoseTool`, `CancelNavigateToPoseTool`, `GetNavigateToPoseFeedbackTool` y `GetNavigateToPoseResultTool`, que LARIC envuelve en su propia `NavigationTool`.

LARIC depende de un fork modificado de RAI (github.com/lourdesfll29-maker/rai). La modificación, en el archivo `tools/ros2/navigation/nav2.py`, corrige dos defectos del tratamiento del resultado de `/navigate_to_pose` (estado no reiniciado entre misiones y `GoalStatus` descartado). El detalle completo está en el Manual de Programación.

3.9. La pila de la HMI: PyQt5, voz y gettext

La interfaz combina, en un único objeto, un nodo de ROS 2 y un widget de PyQt5. Como cada framework tiene su propio bucle de eventos, el ciclo de ROS 2 se integra en el de Qt mediante un `QTimer` que ejecuta `relpy.spin_once` cada 20 ms. La entrada por voz usa Push-to-Talk (barra espaciadora): graba con `sounddevice` y transcribe con la API de Google Speech Recognition (biblioteca `SpeechRecognition`) en el idioma activo. La salida se muestra en un log con código de colores. Toda la i18n se apoya en `gettext`. Además, el sistema responde por voz: el agente publica una frase corta y la HMI la sintetiza y la reproduce en el idioma activo; el audio sale por el PC.

4. Cómo se interconectan los componentes

4.1. Tópicos de ROS 2

Tópico	Sentido	Propósito
<code>/from_human</code>	HMI -> Agente	Texto del comando del usuario (voz o teclado).
<code>/robot_feedback</code>	Agente -> HMI	Mensajes de estado mostrados en el log.
<code>/laric_voice</code>	Agente -> HMI	Frase corta para leer en voz alta (salida por voz)
<code>/emergency_stop</code>	HMI -> Agente	Señal de parada de emergencia (Bool).
<code>/language_changed</code>	HMI -> Agente	Código de idioma para sincronizar la i18n.
<code>/initialpose</code>	RViz/Foxglove -> Agente	Initialization de la localización.
<code>/cmd_vel</code>	HMI/Nav2 -> Robot	Velocidad directa (parada y gesto de negación).

Tabla 1. Tópicos principales de ROS 2 en LARIC.

4.2. Flujo de extremo a extremo

1. El usuario habla (manteniendo pulsada la barra espaciadora) o escribe en la HMI.
2. Si es voz, se graba con sounddevice y se transcribe con Google STT en el idioma activo.
3. La HMI publica el texto (en minúsculas) en `/from_human`.
4. El agente lo recibe en `on_human_input` y lanza un hilo de razonamiento (`think`).
5. LangChain construye el prompt (`SYSTEM_PROMPT` + mensaje) y llama al LLM.
6. El LLM decide qué herramienta invocar y con qué parámetros (tool calling).
7. La herramienta comprueba la localización y el bloqueo de movimiento, y envía el goal al action server de Nav2 correspondiente.
8. Un bucle de sondeo espera el resultado (`GoalStatus`) o una orden de parada.
9. El agente publica el resultado y los mensajes de progreso en `/robot_feedback`.
10. La HMI muestra esos mensajes, con código de colores, en su log.
11. Además, el agente publica una versión corta del mensaje en `/laric_voice` y la interfaz lo reproduce por voz en el idioma activo.

4.3. Flujo de localización

Ninguna orden de movimiento se acepta hasta que el robot está localizado. El operador fija la pose inicial con la herramienta “2D Pose Estimate” de RViz (simulación) o publicando desde Foxglove (robot real); en ambos casos el efecto es un mensaje en `/initialpose`. El agente lo detecta (callback `on_initial_pose`) y activa la primitiva `localised_event`. Durante una breve ventana de gracia tras el arranque se ignoran las poses retenidas por DDS, de modo que cada sesión exige una localización fresca.

4.4. Flujo de parada de emergencia

La parada de emergencia tiene dos capas y evita por completo al LLM para garantizar la mínima latencia. Al pulsar el botón rojo de la HMI (o Ctrl+E), la interfaz:

1. Publica directamente un Twist de ceros en `/cmd_vel`, lo que frena el robot de inmediato.
2. Publica True en `/emergency_stop`. El agente, al recibir esta señal, ejecuta directamente `StopTool`, que activa `abort_event` (corta cualquier movimiento en curso) y cancela el goal de Nav2 activo.

4.5. Sincronización de idioma

La HMI y el agente son procesos distintos, así que cada uno tiene su propio catálogo de traducción. Cuando el usuario cambia de idioma en la HMI, esta actualiza su catálogo y publica el nuevo código en `/language_changed`; el agente lo recibe y llama a `set_language` para que su realimentación pase a ese idioma.

5. El agente y sus 7 herramientas

El agente expone siete herramientas. Cada una valida sus entradas con un esquema Pydantic y comparte tres primitivas de sincronización (`abort_event`, `motion_lock`, `localised_event`). Todas devuelven una cadena con prefijo `SUCCESS / ACTION FAILED / ACTION ABORTED / ERROR` que el LLM evalúa según las reglas del prompt.

Herramienta	Accion Nav2	Función
spin_robot	/spin	Giro en el sitio por un ángulo en grados.
move_robot	/drive_on_heading, /backup	Avance o retroceso una distancia.
execute_sequence	(combina las anteriores)	Encadena varios movimientos en orden.
navigate_to_location	/navigate_to_pose	Navega a una ubicación semántica o a (x, y).
navigate_sequence	/navigate_to_pose (en bucle)	Recorre varias ubicaciones conocidas en orden.
negation_gesture	/cmd_vel	Gesto físico de negación ante peticiones imposibles.
stop_robot	cancel + /cmd_vel	Parada inmediata y cancelación de la misión.

Tabla 2. Herramientas del agente.

Las herramientas de movimiento siguen un patrón común: comprueban la localización, adquieren `motion_lock` de forma no bloqueante, envían el goal al action server y entran en un bucle de sondeo que vigila la finalización, el `abort_event` o el timeout. El detalle de implementación de cada una está en el Manual de Programación; el repertorio de comandos que aceptan, en el Manual de Comandos.

6. Diseño de seguridad

La seguridad no es un añadido, sino un principio transversal del diseño. Se apoya en cuatro mecanismos:

- **Localización obligatoria:** Ninguna herramienta de movimiento se ejecuta sin `localised_event` activo. Garantiza que el robot siempre sabe dónde está antes de moverse.
- **Serialización del movimiento:** `motion_lock` (un cerrojo reentrante) impide dos movimientos simultáneos. `StopTool` no lo adquiere, para poder interrumpir siempre, aunque haya un movimiento en curso.
- **Parada de emergencia en dos capas:** El botón de la HMI publica un Twist de ceros directamente en `/cmd_vel` (efecto inmediato) y, además, notifica al agente por `/emergency_stop` para que cancele los goals de Nav2. Evita por completo el LLM, garantizando la mínima latencia.
- **Razonamiento acotado:** El `AgentExecutor` limita las iteraciones y el prompt prohíbe encadenar herramientas o auto-cancelarse, evitando bucles y acciones no deseadas.

7. Internacionalización

LARIC es bilingüe (español/inglés). La i18n se basa en gettext: los textos visibles se envuelven en la función `_()`, que resuelve el catálogo activo en cada llamada, de modo que un cambio de idioma en caliente surte efecto inmediato en todos los módulos. Como la HMI y el agente son procesos distintos, el cambio se propaga por el tópico `/language_changed`. Las cadenas internas que el LLM evalúa (SUCCESS, ACTION FAILED, etc.) se mantienen deliberadamente en inglés, porque el prompt las reconoce por ese prefijo: traducirlas rompería la evaluación del resultado.

8. Simulación frente a robot real

El mismo código del agente se ejecuta en ambos entornos; lo que cambia es dónde corre Nav2 y qué mapa semántico se usa, seleccionado por los conmutadores `is_real_robot` e `is_from_lab`.

Aspecto	Simulación	Robot real
Plataforma	TurtleBot3 Waffle Pi (Gazebo)	Husarion ROSbot
ROS 2 / Ubuntu	Humble / 22.04	Jazzy / 24.04
Ejecución de Nav2	En el PC o VM	A bordo del robot
Visualización / pose	RViz (2D Pose Estimate)	Foxglove (Publish Pose Estimate)
Destinos del mapa	Habitaciones de <code>turtlebot3_house</code>	Pasillos y zonas de la planta del laboratorio

Tabla 3. Comparativa de los dos entornos.

Durante las pruebas en el laboratorio se observó que el LIDAR, montado a cierta altura en el ROSbot, no detecta las patas de las sillas, los pies de la pizarra ni los pies de las personas, por lo que el robot tiende a chocar con esos obstáculos, y que avanzaba muy despacio en zonas estrechas. Por ello las pruebas se trasladaron al exterior del laboratorio, y el mapa semántico real recoge waypoints de zonas abiertas en lugar de puntos interiores del laboratorio.

9. Decisiones de diseño

- **Todo el movimiento a través de Nav2:** En fases iniciales se valoró una arquitectura reactiva con herramientas de movimiento propias basadas en odometría, que operaba "a ciegas" sin posición en el mapa. Se descartó en favor de delegar todo en Nav2: estandariza la arquitectura siguiendo el patrón oficial de ROS 2, mantiene la coherencia de los costmaps y reutiliza un stack probado. El precio es que todo movimiento exige localización previa.
- **Pose inicial manual en lugar de autolocalización:** Se probó a que el robot se autolocalizara girando 360, pero fallaba en entornos simétricos (habitaciones iguales). Dejar la pose inicial en manos del robot es arriesgado, así que la fija el operador, como es práctica habitual en robótica real.
- **Dos backends de LLM en lugar de uno:** Empezar con un único proveedor (o un modelo local pesado) era frágil: límites de tokens, bloqueos

regionales o falta de potencia. La solución (Ollama del ai2 como vía principal y Groq como respaldo) desacopla el sistema de un único proveedor.

- **Voz propia (entrada y salida) en lugar de la pila de RAI:** La captura y transcripción de voz se implementaron en la HMI (Push-to-Talk + Google STT) en lugar de usar la pila speech-to-speech de RAI, para no añadir cómputo local pesado y mantener un control explícito (el usuario decide cuándo habla), más seguro para un robot. Por el mismo motivo, la respuesta hablada (TTS) se resolvió con edge-tts (voces neuronales de Microsoft), que da una voz más clara y natural que otras alternativas, en lugar de la síntesis de la pila S2S de RAI; el audio se reproduce en el PC, ya que el ROSbot 3 no dispone de altavoz.
- **execute_sequence frente a una única llamada por turno:** LangChain ejecuta una sola herramienta por turno; para soportar órdenes encadenadas ("gira y luego avanza") se empaqueta la secuencia en una sola llamada, en lugar de obligar al LLM a traducir intenciones relativas a coordenadas globales.
- **navigate_sequence frente a encadenar navegaciones a mano:** LangChain ejecuta una sola herramienta por turno; para soportar rutas de varias paradas ("patrulla la cocina y el salón") se recorre una lista de ubicaciones con nombre en una sola llamada, en lugar de obligar al LLM a lanzar una navegación tras otra. A diferencia de execute_sequence, que encadena movimientos primitivos, navigate_sequence encadena ubicaciones del mapa: navega a cada una respetando su posición y orientación y espera unos segundos al llegar antes de continuar.
- **return_direct en las herramientas:** La salida de cada herramienta se devuelve directamente, sin un segundo paso por el LLM, reduciendo latencia y consumo de tokens; solo las respuestas conversacionales pasan por el modelo.

El razonamiento detallado de cómo se llegó a cada una de estas decisiones, con los obstáculos que las motivaron, está recogido en el documento "Evolución del Proyecto".

10. Resultados

El sistema interpreta correctamente comandos de rotación, movimiento, secuencias, navegación semántica, parada y conversación, en español e inglés y en ambos entornos (simulación y robot real), con una parada de emergencia fiable. La portabilidad entre simulación y robot real se logra sin cambios en el código del agente.